

DARK FACTORY · TABLEKEEPER TRACK

Four agents build the software.
None of them may mark its own work.

A band where the separation of powers is a runtime tool policy, not a sentence in a prompt.

THE PROBLEM

Ask an agent whether its own code is correct
and it will tell you yes.

Every pipeline that lets the author sign off their own work has this hole. Most of them ship on **silence**: nobody is assigned to notice the thing that was never checked.

So this factory treats three situations identically, and all three are refusals: a failing verdict, a verdict with no evidence, and **a claim with no verdict at all**.

DIVIDE BY AUTHORITY, NOT BY JOB TITLE

Architect / Developer / Reviewer / QA splits the *work*.
This splits the **power**.

SEAT	MAY CHANGE THE WORK	PRODUCES EVIDENCE	OPENS THE GATE
Scribe	no	no	no
Builder	the only one	no	no
Auditor	no	the only one	no
Registrar	no	no	the only one

Exactly one owner per column, and no seat holds two. In a job-based roster, a reviewer that also fixes what it found is a helpful reviewer. Here it is a contradiction.

NOT A CLAIM. A COMMAND.

The Auditor cannot call an editor.

```
$ band runtime template show --as auditor --json --reveal
scribe      disallowed_tools: ['Edit', 'Write', 'NotebookEdit']
builder     disallowed_tools: [] (none: this seat may change the work)
auditor      disallowed_tools: ['Edit', 'Write', 'NotebookEdit']
registrar    disallowed_tools: ['Edit', 'Write', 'NotebookEdit']
```

`policy.txt` ships in the repository and is generated by `setup.sh`, not written by hand. The seat does not decline to write code. The tool is not there to call.

THE HONEST BOUNDARY OF THAT CLAIM

Bash cannot be denied, so tool policy alone does not enforce the separation.

```
error: owned Claude Code runtimes cannot deny "Bash": Jam pre-approves  
jam CLI commands so room collaboration never waits on a provider prompt
```

A seat with a shell can still write a file, and **ours did**. From the room log, the Scribe, denied every file-editing tool:

“*No file-write tool is exposed in this runtime, so I'll create the ledger with a quoted heredoc.*”

SO WHAT ACTUALLY GUARANTEES IT

The Auditor works in a clone it throws away.

It clones at the revision named in the handoff, runs the check there, and discards it.

IT CANNOT AUDIT UNCOMMITTED WORK

So "it works on my machine" cannot enter the record.

A SHELL DOES NOT HELP IT

Anything it writes goes into a directory that is deleted and was never on the path to the submission.

The property that matters, **that the seat producing evidence cannot change what the evidence is about**, holds whatever tools the seat has.

Is this just CI with extra steps?

About enforcement, that objection is **correct**, and this design does not pretend otherwise. A required status check on a protected branch cannot be bypassed and needs no agents.

What a required check cannot do is the rest of the loop. A failing check emits a red cross and stops. It does not decide whether the claim or the implementation was wrong, it cannot route the failure to whoever owns that authority, and it has one runner with one capability set rather than four participants with deliberately different ones.

A gate enforces. A band negotiates. Remove Band and you keep the enforcement and lose the loop.

MEASURED OUT OF THE ROOM EXPORT, NOT WRITTEN BY US

Autonomy and collaboration.

8,292

messages in the submitted
room

1

human message. The
dispatch, and nothing else

213

verdicts committed before
they were announced

140

commits, none of them hand-
written by a person

Tool calls split 25.9 / 25.2 / 24.6 / 24.4 across Registrar, Scribe, Builder and Auditor.

The rules warn that "one seat carrying 90% of it looks the same however many messages it sent". This is the inverse.

THE GATE, WORKING

Every refusal is measured against the grader.

```
# Verdict on C-0: FAIL
Audited in: .auditor-clones/C-0 (throwaway clone)

Exit: 1
curl: (7) Failed to connect to 127.0.0.1 port 18080
NOT HEALTHY WITHIN 60s

## Graded suite at this revision: FAIL
26 failed, 1 passed, 93 errors
```

On every failure the Auditor re-runs the graded suite at the same revision, so **"would this have failed anyway"** is two recorded runs, not an opinion.

WHAT THE FACTORY SAID ABOUT ITSELF

*“This ledger is entirely unknown.
The gate has not yet been shown to pay for itself.”*

The Registrar, closing the submitted run

Eleven of twelve refusals were the gate finding fault with **its own specification** rather than the product. The one real implementation defect was caught during construction, and no refusal records it.

It could have claimed twelve saves. It counted them, found it could not evidence one, and wrote that down instead.

WHAT RUNNING IT TAUGHT US THAT REVIEW DID NOT

Four things broke. Each changed the design.

THE FLAW

WHAT CHANGED

Bash cannot be denied. The runtime refuses to start without a shell

Enforcement demoted to a speed bump; the discarded clone is the guarantee

Verdicts died with a session. After a restart the Auditor could not resend evidence it had produced, and the Registrar correctly refused the paraphrase

Verdicts are committed to a file before they are announced

Nobody owned the queue. All three seats obeyed their mandates and the work still stopped. This is the characteristic failure of a clean separation

The Builder self-dispatches; going idle is named as a misreading

A silent stall looks like a healthy factory. Seats reported themselves live while processing nothing for ninety minutes

A watchdog that counts real work, not heartbeats

MEASURED WITH BAND USAGE ROOMS

The ceremony is not free.

\$91

Scribe

\$88

Builder

\$67

Registrar

\$63

Auditor

List-price estimates, **not a bill**: the seats run on a subscription, so no money changed hands.

The ledger keeper outspent the builder. A large share of this run went on the band auditing its own bookkeeping rather than the product, and that is the clearest cost of the design.

WHAT THE FACTORY SHIPPED

Stage 1 and stage 2, both claimed.

```
stage 1: pass
stage 2: pass
stage 3: fail
highest contiguous stage: 2
claimed stage: 2 on the shipped checks
```

harness check reports zero problems: gates 1, 2 and 4 pass, and gate 3 is proven by the run above. 120 of 120 stage-1 checks, all of suite 2, and a front end that holds at a 375-pixel viewport with no horizontal scroll.

STAND IT UP ON A DIFFERENT PROBLEM

Nothing in mandates/ names this track.

```
OWNER=you ./setup.sh
```

```
Four seats created and bound. Effective policies written to policy.txt.
```

That is a rule of the event, and the honest test of whether this is a factory or a transcript of one problem.

github.com/Aashan47/fail-closed